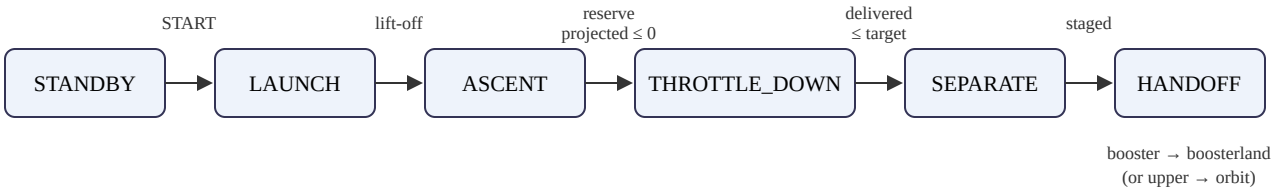
ascent — how it works

MechJeb's ascent autopilot flies the climb. This program decides when the booster separates: at the last moment it can still fly home. It throttles the booster down so the upper stage pulls away from it, stages, and hands the booster to `boosterland`. `ascent/autopilot.py` is the phase machine and the only kRPC-aware code. `ascent/budget.py` is the pure model. Every number is a field of `ascent/config.py`.

Phases



phase	does	leaves when
STANDBY	Waits for START (or `--autostart`). Sets MechJeb's ascent: `ASCENT_TYPE` (1 = PVG), periapsis *and* apoapsis both `ORBIT_ALT_M`, `INCLINATION_DEG`, autostaging off, and engages it. The throttle is MechJeb's throughout; this program never writes it on the ascending vessel (kRPC would hold any value written).	START pressed.
LAUNCH	Stages until an engine is active, then waits for lift-off. If the vehicle is lit but still on the pad after `LAUNCH_CLAMP_WAIT_S`, it releases the next stage (clamps). It never stages a stage containing a decoupler.	An engine is active and the vehicle is neither pre-launch nor landed. It then reads the stack (below).
ASCENT	MechJeb flies. Every tick the program re-runs the recovery budget for the present state and projects the reserve `SEP_LEAD_S` ahead at its present rate of fall.	The projected reserve ≤ 0 above `MIN_ALT_M`, or the booster is out of propellant.
THROTTLE_DOWN	Sets the booster engines' thrust limits so its thrust is the separation target. MechJeb keeps the main throttle.	Delivered booster thrust ≤ target·(1+`SEP_THRUST_TOL`) on `SEP_CONFIRM_TICKS` ticks in a row, or after `SEP_THROTTLE_TIMEOUT_S` (logged as TIMEOUT).
SEPARATE	Stages down to the booster's decoupler. Finds the booster as the vessel its engines now belong to and sets its main throttle to 0. The thrust limits stay on.	Immediately.

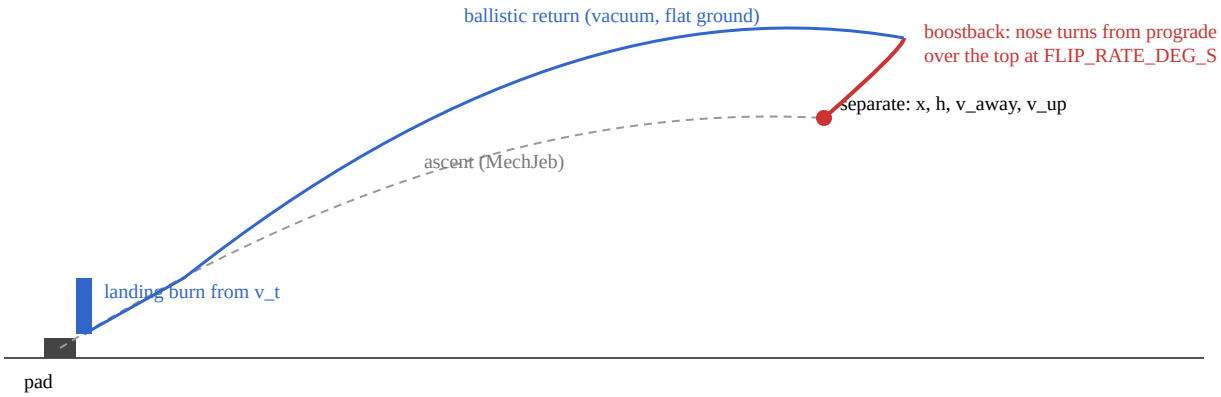
HANDOFF	`HANDOFF=booster`: makes the booster the active vessel and runs `boosterland.Autoland` in this process, writing its own LOG named in this one. After boosterland's first tick, the limits are released unless its engine cluster owns them (more than one group). Logs the propellant the booster lands with. `HANDOFF=upper`: turns MechJeb's autostaging back on and stays until MechJeb disengages (or `ORBIT_TIMEOUT_S`, 3600 s). It logs an `UPPER` line every `LOG_INTERVAL_UT`: apoapsis, periapsis, upper-stage thrust, mass and MechJeb's status.	Boosterland finishes / MechJeb disengages.

Reading the stack

Nothing about the vehicle is configured.

- **Booster**: the parts dropped at the highest decouple stage among the lit engines.
- **Its engines**: the liquid engines on those parts.
- **Upper-stage engines**: every other engine that is lit now, or that the same staging event activates.
- **Booster mass** = dry + LiquidFuel + Oxidizer in that decouple stage. The dry part is measured once at lift-off; only the propellant changes after that.
- **Booster engine figures**: thrust and Isp in vacuum and at the pad's pressure.
- **Radius**: from the widest propellant tank's own bounding box.
- **Engine groups**: `boosterland.engines.group_engines`, in the plane perpendicular to the engines' mean thrust direction. The vessel frame is the root part's and need not lie along the booster's axis.

The recovery budget (`budget.recovery`)



The booster's path if it separated now. All of it is recomputed every tick from the present state.

Boostback. A finite burn integrated in 0.1 s steps in the vertical plane through the pad. The nose starts along the velocity and turns over the top toward the pad. The turn is flown the way boosterland will fly it. Ascent reads boosterland's configuration, including `--booster-set`, to know which flip that is:

- **`FLIP_THROTTLE_LAW`** (the default): each step takes the throttle from boosterland's own `guidance.flip_throttle`. The turn is a bang-bang rotation at u·gimbal + RCS/wheel angular acceleration, until the nose is within boosterland's `FLIP_ALIGN_DEG`. The booster's authority is computed from its own parts, summed in the booster's axis frame and refreshed every `AUTHORITY_REFRESH_S`:
  - centre of mass;
  - pitch inertia: point masses, plus a solid cylinder for each tank;
  - gimbal torque: thrust × sin(gimbal range × limit) × lever;
  - RCS: thrust × lever / π;
  - wheel torque.
- **Full-throttle flip**: a constant turn at `FLIP_RATE_DEG_S`.

The engines thrust along the nose at the throttle times the booster's full vacuum thrust, with its mass falling at that thrust/(Isp·g0). At each step the required horizontal speed toward the pad is x/T, where $T = (v_{up} + \sqrt{(v_{up}^2 + 2gh)})/g$ is the vacuum fall time from the current height. g is taken at that height. The burn ends when the horizontal velocity reaches that speed. Any cross-track velocity is added to the burn in quadrature.

Landing burn. The apex follows from the burn-out state. The fall speed v_t is the smaller of two values:

- the vacuum speed of a fall from the apex;
- terminal velocity $\sqrt{(2mg / (\rho_{pad} \cdot$ `LANDING_CD` $\cdot \pi r^2))}$.

The burn is $dv = v_t \cdot a/(a-g)$, where a is the pad-pressure thrust over the landing mass. It is evaluated twice: the second pass uses the mean mass over the burn.

Propellant left = mass after both burns − dry mass. The boostback is spent at vacuum Isp and the landing burn at the pad's Isp. **Need** = propellant now − propellant left. **Reserve** = left − `RESERVE_FRACTION`·need. Separation starts when the reserve, projected `SEP_LEAD_S` ahead, reaches zero.

The throttle-down

The main throttle is read from the booster's engines once, as their delivered thrust ÷ the thrust their limits allow (`control.throttle` does not show MechJeb's throttle). The upper stage's acceleration is a_u = Σ(max thrust·limit of its engines)·main throttle / (vessel mass − booster mass). The booster's target thrust is `SEP_ACCEL_RATIO`$\cdot a_u \cdot$booster mass, so that once free its acceleration is 0.75 of the upper stage's. The demand (target / the booster's present full thrust at the main throttle) goes to `boosterland.engines.allocate`:

- groups fill in lighting order: the centre engine first, then symmetric pairs outward, each group to full before the next lights;
- so the thrust comes off the outer pairs first;
- each engine's share is written as its thrust limit.

The phase stages only when the booster's *delivered* thrust is under the target.



e.g. five engines, demand 0.27:
centre 1.00, first pair 0.18, second pair 0.00
booster free: 0.75 × upper stage acceleration
→ the upper stage pulls away; boosterland then cuts the booster to 0

Configuration

field	default	role
`RESERVE_FRACTION`	0.15	Propellant kept beyond the modelled burns, as a fraction of them. Covers correction burns, throttling losses in the landing burn, Vernor RCS, and drag on the boostback.
`FLIP_RATE_DEG_S`	30	Turn rate of the full-throttle flip, used only when boosterland flies without `FLIP_THROTTLE_LAW`
`AUTHORITY_REFRESH_S`	15 s	How often the booster's inertia and torque are re-summed over its parts
`LANDING_CD`	1.0	Drag coefficient on the booster's frontal area for the terminal velocity
`SEP_ACCEL_RATIO`	0.75	Booster's free acceleration as a fraction of the upper stage's
`SEP_LEAD_S`, `MIN_ALT_M`	2 s, 1000 m	Look-ahead on the reserve; no decision below this height
`HANDOFF`	booster	What is flown after separation

Requires MechJeb 2.15 and KRPC.MechJeb built for it (darchambault's fork, v0.8.1) in `GameData/kRPC/`.